

You are here / [🏠](#) / etc / opencv / opencv

[\[\[etc:opencv:opencv \]\]](#)

OpenCV optimization using OpenCL

CMake

OpenCL is the default option.

If you see the message below while running CMake, you can use OpenCL. (See OpenCV installation.)

```
...
--
--   OpenCL:                               YES (no extra features)
--   Include path:                         /home/odroid/opencv/3rdparty/in
clude/opencv/1.2
--   Link libraries:                       Dynamic load
--
...
```

If not, add `-DWITH_OPENCL=ON` option when executing CMake as shown below.

```
$ cmake ..\
  -DCMAKE_BUILD_TYPE=Release \
  -DOPENCV_GENERATE_PKGCONFIG=ON \
  -DBUILD_PERF_TESTS=OFF \
  -DOPENCV_EXTRA_MODULES_PATH=$HOME/opencv_contrib/modules \
  -DWITH_OPENCL=ON
```

Link

In general, the path to the OpenCL library is

- ARM-32bit: `/usr/lib/arm-linux-gnueabi/libOpenCL.so`
- ARM-64bit: `/usr/lib/aarch64-linux-gnu/libOpenCL.so`

If you have the library in this location, it is automatically linked.

If not, install **mali-fbdev** and find libOpenCL or libMali and make a symbolic-link to the above path.

```
$ sudo apt install -y mali-fbdev &&\
  sudo find / -name "libOpenCL*" -o -name "libMali*"
[a found path]
...
```

```
$ sudo ln -s [the found path] [the above path]
```

Information

Install **clinfo** to easily check the platform information.

```
$ sudo apt install -y clinfo ocl-icd-libopencl1 &&\
  sudo mkdir -p /etc/OpenCL/vendors &&\
  sudo touch /etc/OpenCL/vendors/mali.icd
```

Add the path you saw in the Link section to **mali.icd**

clinfo reads .icd file and prints platform information.

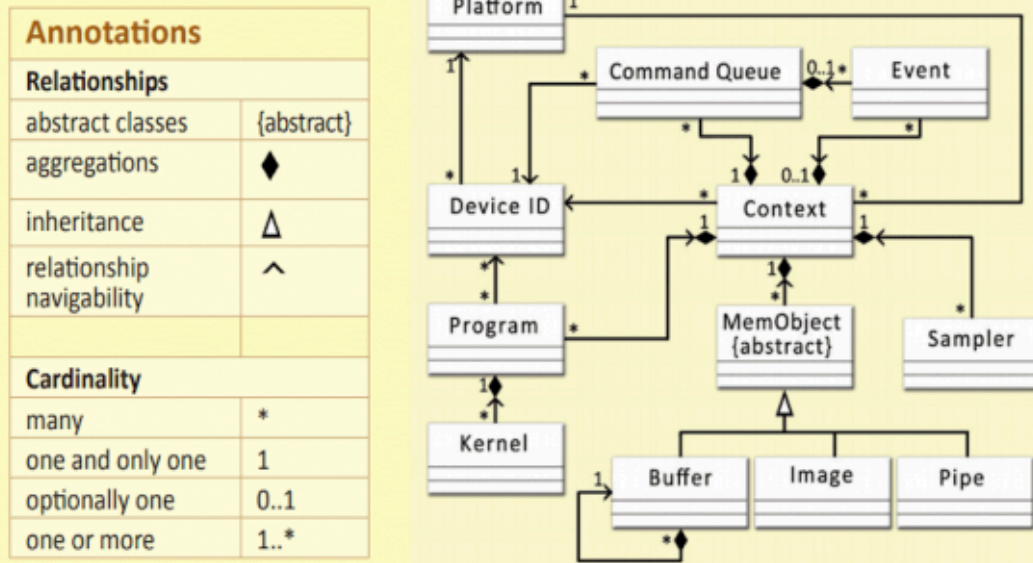
```
$ clinfo
Number of platforms          1
  Platform Name              ARM Platform
  Platform Vendor            ARM
  Platform Version           OpenCL 1.2 v1.r17p0-01rel0.a881d28363
cdb20f0017ed13c980967e
  Platform Profile           FULL_PROFILE
...
```

Simple OpenCL programming sequence

Before you use OpenCL to optimize OpenCV, it's good to know the OpenCL programming sequence.

OpenCL Class Diagram

The figure below describes the OpenCL specification as a class diagram using the Unified Modeling Language¹ (UML) notation. The diagram shows both nodes and edges which are classes and their relationships. As a simplification it shows only classes, and no attributes or operations.



¹ Unified Modeling Language (<http://www.uml.org/>) is a trademark of Object Management Group (OMG).

1. Write kernels.
2. Get `cl_platform_ids`.
3. Select a `cl_platform_id` and get `cl_device_ids` in the platform.
4. Select a `cl_device_id` and create a `cl_context`.
5. Create a `cl_command_queue`.
6. Create a `cl_program` using a kernel.
7. Build the `cl_program`.
8. Create a `cl_kernel` from the `cl_program`.
9. Create `cl_mems(buffer)`.
10. =====loop=====
11. Set kernel arguments.
12. Enqueue the kernel.
13. Wait until it finish.
14. Get kernel arguments(return value)
15. ...

CPU vs. GPU

- Hardware: Odroid-XU4
- OS: Ubuntu minimal 18.04
- OpenCV: 4.1.0
- OpenCL: 1.2 (full profile)

Test code

In general, Computing with OpenCL(GPGPU) is known to be fast. For testing, I executed **the Sobel function four times** sequentially on the CPU and GPU each. Below is the test code.

CPU	GPU
-----	-----

mali_opencv.cpp

```
#include "opencv2/opencv.hpp"
#include "opencv2/core/ocl.hpp"
#include <iostream>
#include <unistd.h>

using namespace std;
using namespace cv;

int main( int argc, char *argv[] )
{
    int64_t start, end;
    float   time_sec;

    start = getTickCount();
    /**
     * gets default platform
     * gets device list
     * selects first device
     * creates context
     */
    ocl::Context context;
    if( !context.create( ocl::Device::TYPE_GPU ) )
    {
        cerr << "Failed to create context" << endl;
        return -1;
    }

    ocl::setUseOpenCL( true );

    int  img_size = atoi( argv[1] );
    UMat u_img( Size( img_size, img_size ), CV_8UC1, USAGE_ALLOCATE
_HOST_MEMORY );
    UMat u_img_2;

    end      = getTickCount();
    time_sec = ( end - start ) / getTickFrequency();
    cout << time_sec << endl;

    start = getTickCount();
    Sobel( u_img, u_img_2, -1, 1, 0 );
    end      = getTickCount();
    time_sec = ( end - start ) / getTickFrequency();
    cout << time_sec << endl;
    u_img_2.release();

    usleep( img_size * 100 );

    start = getTickCount();
```

```
Sobel( u_img, u_img_2, -1, 1, 0 );
end      = getTickCount();
time_sec = ( end - start ) / getTickFrequency();
cout << time_sec << endl;
u_img_2.release();

usleep( img_size * 100 );

start = getTickCount();
Sobel( u_img, u_img_2, -1, 1, 0 );
end      = getTickCount();
time_sec = ( end - start ) / getTickFrequency();
cout << time_sec << endl;
u_img_2.release();

usleep( img_size * 100 );

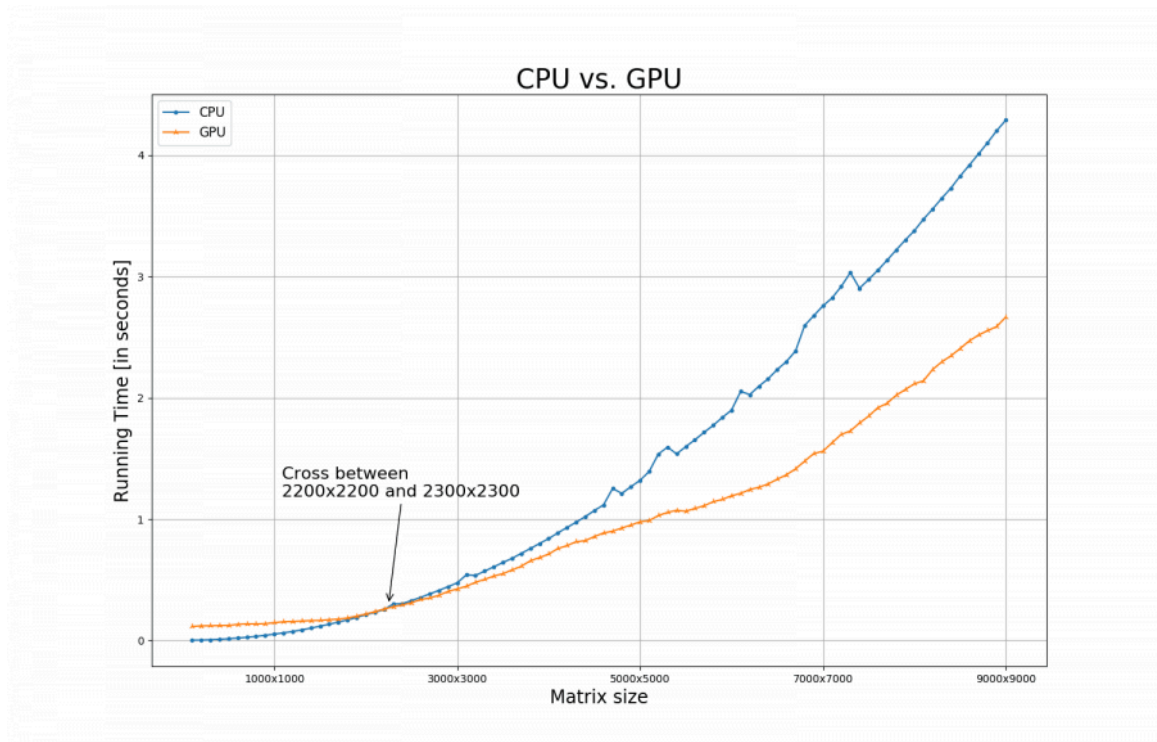
start = getTickCount();
Sobel( u_img, u_img_2, -1, 1, 0 );
end      = getTickCount();
time_sec = ( end - start ) / getTickFrequency();
cout << time_sec << endl;
u_img_2.release();

u_img.release();

return 0;
}
```

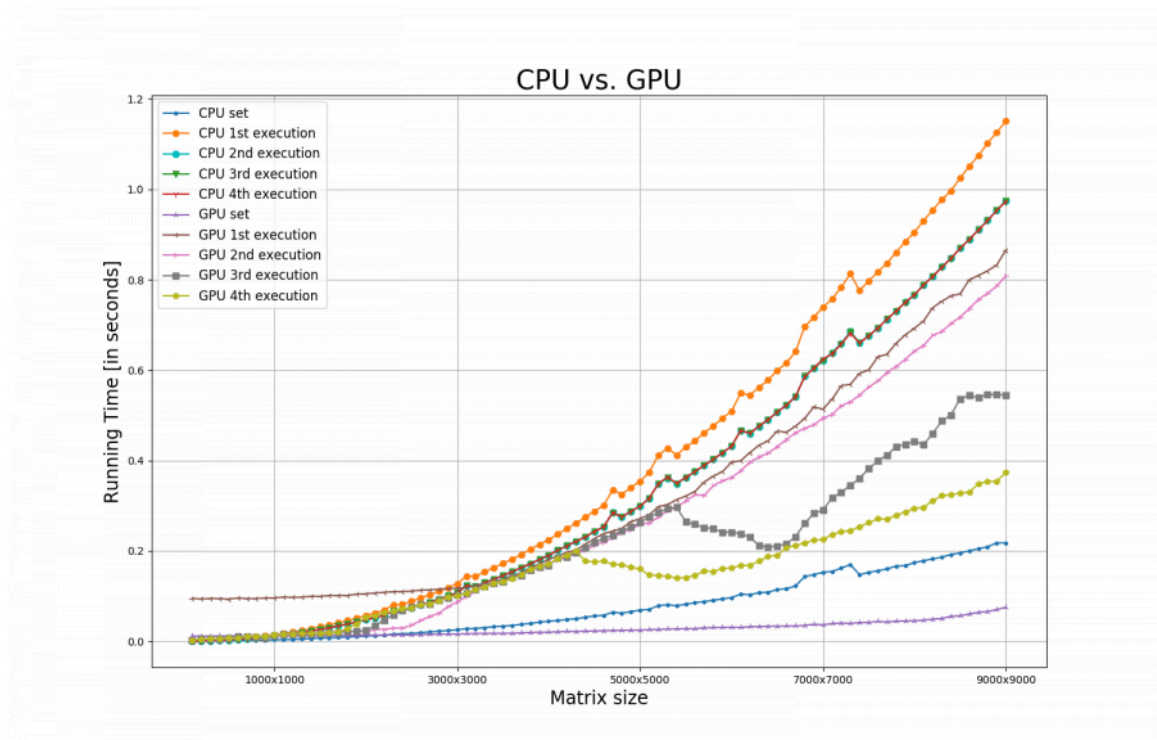
Test result and Analysis

Total running time



Total running time seems to be depending on the matrix size. I checked the individual running time for further analysis.

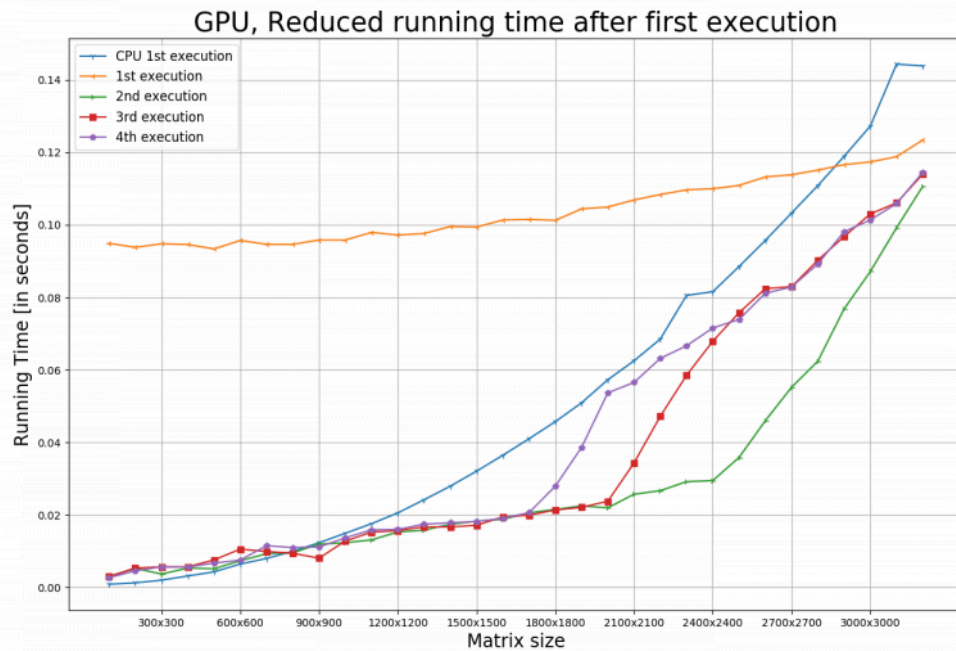
Individual running time



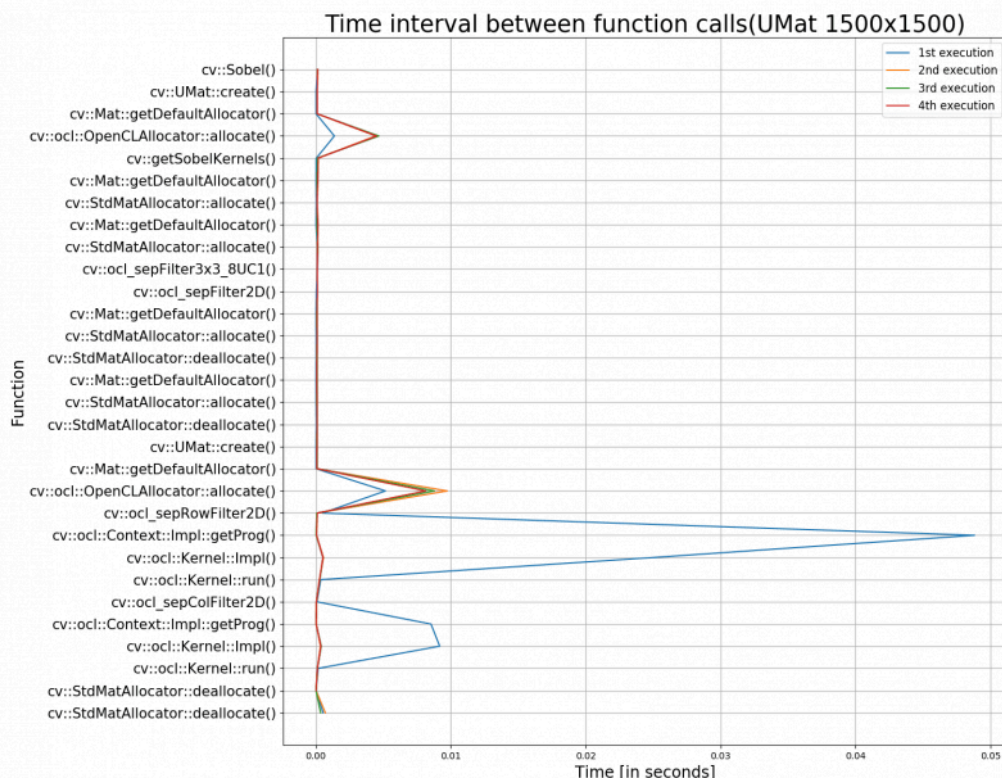
When computing in the CPU, the running time depends on the matrix size.

When in the GPU, There were interesting aspects.

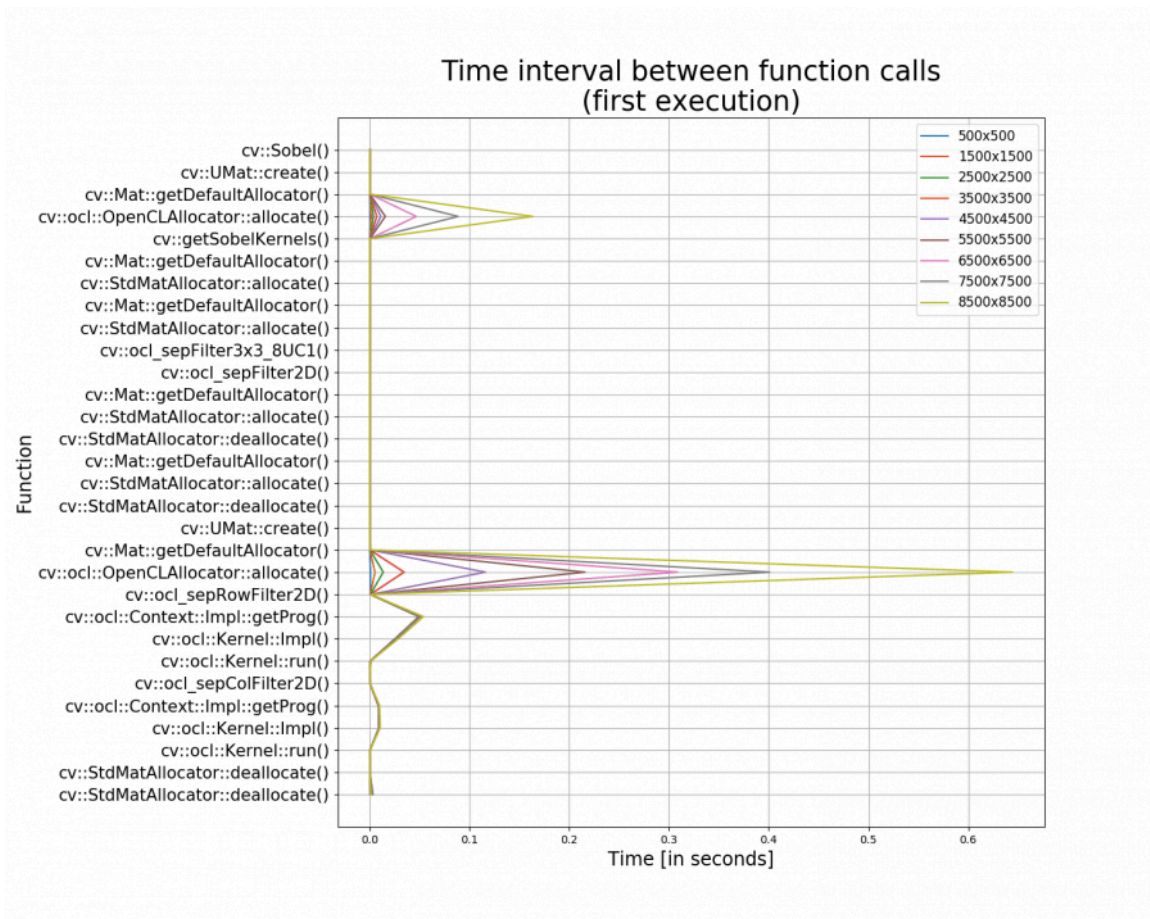
- the 1st execution with the matrix size under 3000×3000 .
- the 3rd and 4th execution with the matrix size over 5000×5000 .



When using the matrix whose size is between 1000×1000 and 3000×3000 , after the first execution, Computing in the GPU is faster than in the CPU.



The y-axis represents the sequence of function calls from top to bottom. Because building a program and creating a kernel, the first execution consumes more time than the other executions. (see the **Simple OpenCL programming sequence** section)



The larger the matrix size, the more the `cv::ocl::OpenCLAllocator::allocate()` time consumption increases.

Conclusion

Computing in the GPU is much faster than in the CPU. But setup time and memory allocating time are so long.

For optimization

- When a matrix size is large enough, GPU is faster.
- When calling a function repeatedly, GPU is faster even if the matrix size is not large enough.
- If a function using the CPU is between functions using the GPU, change from CPU to GPU.
- If you tested your application in X window, You should turn off X window or use console boot mode when you use it.
- If you need more optimization, use OpenCL directly without OpenCV.

Mali GPU is different from ordinary desktop GPU. Especially the Memory model is different.

If you need more optimization, you should read ARM® Mali™ GPU OpenCL Developer Guide.

ODROID Wiki



Except where otherwise noted, content on this wiki is licensed under the following license:
CC Attribution-Share Alike 4.0 International